Design-Space Characterization of Layer-Streamed Neural Network Inference on the ATmega328P for Resource-Constrained ASL Landmark Classification

Al Mahmud Samiul

Department of Electrical and Electronic Engineering,

Faridpur Engineering College, Faculty of Engineering and Technology,

University of Dhaka,

Faridpur, Dhaka

Abstract

Eight-bit microcontrollers such as the ATmega328P impose simultaneous SRAM, flash, and compute constraints 2 KB of RAM and no hardware floating-point unit which block direct deployment of most neural classifiers. This work presents a resource-aware, layer-streamed MLP inference strategy in which each weight is read directly from PROGMEM at the moment it is needed for a multiply-accumulate, rather than a layer's weights being staged into SRAM ahead of computation, and evaluates it through a controlled design-space study on identical ATmega328P hardware. A streaming-ablation experiment shows this is not an optional optimization; a conventional SRAM-resident baseline is rejected by the compiler or crashes on hardware at every one of five tested MLP topologies (869–4,997 parameters), establishing layer-streaming as the only one of the two evaluated weight-staging strategies that executes successfully across the tested topology range. Across float32, int16, and int8 weight precision, int16 matches float32 accuracy exactly (89.58%) at 10.66 ms per inference a 4.7× speedup over float32's 50.59 ms while int8 reaches 88.45% accuracy at 10.24 ms, trading 1.1 accuracy points for a further 47% reduction in as-built model flash; int16 also preserves a live confidence distribution closely matching float32's, while int8 shows a lower mean confidence and wider spread disproportionate to its accuracy loss. A five-seed replication of the topology sweep shows that narrower topologies are prone to catastrophic training collapse (to near-chance accuracy in 1–2 of 5 seeds), overturning a single-seed observation that a smaller topology outperformed the primary model and instead identifying two wider topologies as more reliable, lower-variance improvements over it. End-to-end timing shows host-side perception, not on-device inference, dominates total system latency, and a Pareto analysis across seven configurations using five-seed mean accuracy for topology comparisons identifies the balanced accuracy–latency–memory operating point. A five-letter (A–E) ASL hand-landmark classification task, evaluated on a single subject, serves as the representative validation workload.

Keywords: embedded machine learning, layer-streamed inference, neural network quantization, design-space exploration, sign language recognition

1. Introduction

Automatic sign language recognition (SLR) can narrow the communication gap between deaf or hard-of-hearing individuals and the wider community, yet most reported SLR pipelines assume a GPU-equipped host or a capable embedded SoC to run the recognition model [8, 9]. This dependency limits deployment to relatively expensive or power-hungry platforms and works against the goal of low-cost, standalone assistive hardware. The ATmega328P at the core of the Arduino Nano represents the opposite extreme of the embedded spectrum: 2,048 B of SRAM, 32,768 B of flash (30,720 B usable after the bootloader), a 16 MHz clock, and no hardware floating-point unit. These constraints

Email address: almahmudsamiul@gmail.com (Al Mahmud Samiul)

are severe enough that the central engineering question is not *which* network architecture to deploy, but *whether any given architecture can be executed on this hardware at all*, and if so, under what execution strategy.

This paper treats that question as a systems research problem rather than a single-application demonstration. Rather than presenting one trained model on one microcontroller, it characterizes the feasible design space of a resource-constrained execution strategy layer-streamed inference, in which each weight is read directly from PROGMEM at the point of use during a dense layer's computation instead of the layer's weights being staged into SRAM beforehand across weight precision (float32, int16, int8) and network topology (five MLP sizes spanning 869 to 4,997 parameters). A five-letter ASL hand-landmark classification task, fed by MediaPipe Hands running on a host PC, serves as the representative validation workload throughout, but the paper's contribution is the characterization itself which combinations of precision and topology are executable on a 2 KB-SRAM, FPU-less device, and what each combination costs in latency, memory, and prediction confidence. On-device classification is the scope of the "real-time" claim made throughout this paper; Section 7.6 makes this scope explicit by decomposing total system latency.

1.1. Motivation

Four observations motivate this design-space framing. First, whether a weight-staging execution strategy is even viable on the ATmega328P is an empirical question, not an assumption; Section 5 shows that a conventional SRAM-resident baseline is rejected by the compiler or crashes on hardware at every topology size tested, which reframes layer-streaming from "a memory optimization we chose" to "the only evaluated weight-staging strategy that executed successfully across the tested topology range." Second, per-neuron symmetric int16 quantization retains floating-point accuracy essentially without loss, at twice the flash footprint of int8, while int8 the precision most commonly assumed by default in TinyML deployments [5] is not accuracy-neutral, so the choice between them is a genuine design decision rather than a fixed convention. Third, quantization's effect on live prediction *confidence* is a separate axis from its effect on accuracy, and one that most TinyML evaluations do not report. Fourth, on-device inference latency is only one term in total system latency; isolating it from host-side perception and serial communication is necessary before any claim about "real-time" operation is meaningful.

1.2. Research Questions

This paper is organized around one primary and four secondary questions.

Primary: What precision and execution strategy provides the best accuracy–confidence–latency–memory operating point for neural inference under the SRAM and flash constraints of an 8-bit ATmega328P?

Secondary: (i) Does layer-streaming provide a measurable SRAM benefit over conventional weight staging, or is it strictly necessary for feasibility? (ii) When, if ever, does int8 provide a meaningful systems-level benefit over int16 beyond flash savings? (iii) How does MLP topology affect the feasible operating region under this device's resource budget? (iv) Does weight quantization alter prediction confidence differently than it alters classification accuracy?

1.3. Contributions

This paper makes the following contributions:

1. A streaming-ablation experiment establishing that layer-streamed execution is not merely more memory-efficient than conventional SRAM-resident weight staging, but the only evaluated weight-staging strategy that executed successfully across five evaluated MLP topologies (869–4,997 parameters) on the ATmega328P;
2. A controlled float32/int16/int8 comparison on identical hardware and firmware, jointly measuring accuracy, argmax agreement, live prediction confidence, per-layer latency, flash, and SRAM;
3. A topology-scaling study across five architectures, replicated over five independent training seeds per architecture, establishing both the feasible accuracy–latency–memory design space under this device's resource budget and a seed-fragility failure mode in narrower topologies not observed in the original single-seed sweep;
4. A confidence analysis separating live streaming confidence distributions from test-set bucketed reliability, both computed for all three precisions, showing that int16 preserves a confidence distribution and reliability pattern close to float32's while int8 exhibits a lower mean confidence, greater dispersion, and consistently (though not qualitatively differently) lower bucket-level reliability;
5. An end-to-end latency decomposition separating host-side perception, serial communication, and on-device inference, showing which stage actually dominates total system latency; and

6. A Pareto analysis across seven precision/topology configurations identifying the resulting accuracy–latency–memory trade-offs relevant to deployment.

Consistent with the scope of the underlying dataset (Section 6), all claims in this paper are stated for single-subject, five-letter (A–E) hand-landmark classification.

2. Related Work

2.1. Landmark-Based Hand Perception

MediaPipe Hands provides 21 three-dimensional landmarks per detected hand from a single RGB frame and has become a common front end for gesture and sign-based interfaces because it removes the need to train a dedicated hand detector [1]. In the present pipeline, MediaPipe runs entirely on the host machine; only the resulting normalized (x, y) coordinates are transmitted onward, which keeps the microcontroller free of any vision workload and, as Section 7.6 shows, means host-side perception rather than on-device inference dominates end-to-end latency.

2.2. Quantization and TinyML Inference on Microcontrollers

Post-training integer quantization is the standard technique for fitting neural networks into kilobyte-scale memory budgets. Jacob *et al.* established the integer-arithmetic-only inference scheme most TinyML toolchains build on [3], and Gholami *et al.* survey the broader design space of quantization granularity per-tensor versus per-channel scaling, symmetric versus asymmetric ranges that this paper's per-output-neuron symmetric scheme sits within [4]. TensorFlow Lite for Microcontrollers operationalized this as a widely used interpreter-based runtime and popularized int8 as the default deployment precision [5], a convention the MLPerf Tiny benchmark subsequently standardized latency and accuracy reporting around [6]. A recent survey of quantization methods specifically for microcontrollers catalogs the int8-versus-int16 trade-off, noting that int16 arithmetic recovers accuracy lost to aggressive rounding at a modest, often acceptable, memory cost for small models [7] consistent with this paper's finding that int16 matches float32 on accuracy and produces a closely matching confidence distribution. Most TinyML deployment work, however, adopts int8 as a fixed default and reports a single operating point at a single model size; the present work instead measures both int8 and int16 across five topologies on the same firmware and hardware, treating precision and architecture jointly as tunable design parameters rather than fixed conventions.

2.3. Memory-Aware and Streamed Execution Strategies

A separate line of concern from *what precision* to use is *how weights are staged* during execution. The conventional pattern in embedded neural inference is to copy a layer's weight matrix from program memory into a SRAM working buffer before computing on it, trading flash-read latency for SRAM-read speed. On a device with only 2 KB of SRAM, this pattern's viability is not guaranteed to hold as model size grows. Low-RAM MLP deployment on AVR-class microcontrollers has previously been demonstrated using alternative memory-reduction strategies. Singh [15] demonstrated an MLP deployment methodology on the ATmega328P (MNIST classification, 716 parameters) that partitions the trained model into sequentially executed, layer-wise submodels to keep only one submodel resident in RAM at a time, reducing peak RAM from 2.8 kB to 1.3 kB, and separately discusses direct PROGMEM-based weight access as an alternative execution approach. Earlier work by Izotov *et al.* [16] likewise demonstrated MNIST-digit classification with the LogNNet reservoir network on a 2 KB-SRAM Arduino Uno (ATmega328P). These studies establish the feasibility of neural inference under the ATmega328P's memory constraint, but neither isolates weight-staging strategy as a controlled independent experimental variable swept across multiple MLP topologies with a measured compiler/hardware feasibility boundary, nor jointly characterizes execution strategy, precision, topology, latency, flash, SRAM, and prediction confidence on identical hardware; existing TinyML memory-footprint discussions more broadly [5, 6, 7] report the memory a runtime requires but do not treat weight-staging strategy as an experimental variable in this sense either. In short, I did not find a study that isolates weight-staging strategy as an independent experimental variable and measures its feasibility boundary across multiple MLP topologies on identical ATmega328P hardware; the present work addresses this specific characterization gap. The contribution is not a new mechanism for reading weights from flash; it is the controlled characterization of weight-staging feasibility and its interaction with precision and topology on identical ATmega328P hardware. Section 5 presents a streaming-versus-conventional ablation spanning five topologies as the paper's contribution on this point.

2.4. Sign Language Recognition on Constrained Hardware

Most SLR systems that reach high accuracy rely on convolutional or transformer backbones executed on a workstation or a mobile SoC. Xia *et al.* deployed a MobileNet-YOLOv3 recognizer for a deaf-mute medical consultation kiosk, reporting 90.77% accuracy on an embedded terminal, but their target hardware remains a general-purpose embedded computer rather than an 8-bit MCU [8]. Lim *et al.* combined a lightweight transformer with a large language model on the Pepper robot platform to support socially aware sign-language interaction, again assuming a compute-capable host [9]. Closer to the microcontroller regime, a recent TinyML study introduced a factorized spatiotemporal architecture (S3D-Conv1D) explicitly designed for word-level SLR under strict memory and int8-quantization constraints, reporting the feasibility of MCU-class deployment for video-based gesture recognition [10], though it evaluates a single quantization precision rather than a precision/topology design space. Wearable approaches such as an ESP32-based smart glove pair flex-sensor and inertial data with an LSTM network, offloading recognition to a host computer over Wi-Fi while the microcontroller only aggregates sensor streams [11]. A 2026 ESP32-based study more closely matches the landmark-processing pattern, using MediaPipe's 42 hand-landmark coordinates and a lightweight MLP for alphabet recognition; its ESP32 platform and broader alphabet scope further distinguish that application-oriented result from the present ATmega328P feasibility and design-space characterization [12]. In contrast to these systems, the present work performs the entire classification step not merely sensor aggregation on an ATmega328P, an order of magnitude more memory-constrained than the ESP32-class devices used above, and characterizes a design space (precision × topology × execution strategy) rather than reporting a single operating point.

Table 1: Positioning relative to prior work

Work	Target platform	Task	Precision(s) evaluated	Exec.-strategy variable?
Xia et al. [8]	General embedded terminal	SLR (medical kiosk)	Not MCU-quantized	No
Lim et al. [9]	Pepper robot (host-capable)	SLR + LLM interaction	Full precision	No
S3D-Conv1D [10]	MCU-class	Word-level SLR	int8 (single)	No
Smart glove [11]	ESP32 (sensor only)	SLR (host-offloaded)	Host-side	No
Singh [15]	ATmega328P (2 KB SRAM)	MNIST digits	Quantized (single)	No (layer-wise partitioning, not swept)
This work	**ATmega328P (2 KB SRAM)**	**Landmark SLR (A–E)**	**float32/int16/int8**	**Yes**

3. System Architecture

The pipeline separates perception from classification across two devices connected by a serial link, as illustrated in Fig. 1. This division of labor is deliberate MediaPipe's hand-detection and landmark-localization models are far too large for the ATmega328P's flash and SRAM budget, so perception is offloaded to a host machine, and only the already-compact, pose-normalized 42-dimensional feature vector is transmitted to the microcontroller. The scope of "real-time" claimed in this paper is therefore on-MCU classification of already-extracted hand-landmark features, not end-to-end perception; Section 7.6 decomposes total system latency to make this boundary explicit rather than implicit.

3.1. ATmega328P Resource Budget

Table 2 states the resource envelope that every configuration evaluated in this paper is measured against. The bootloader reserves 2,048 B of the chip's 32,768 B flash, leaving 30,720 B available to application firmware [13]; all flash percentages reported in this paper are relative to that 30,720 B usable budget rather than the raw chip capacity. All 2,048 B of SRAM is available to the application, shared between global static allocations (weight/bias/scale constants staged for read, activation buffers, the serial packet buffer) and the runtime stack.

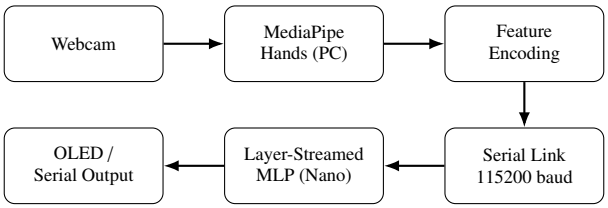

Figure 1: End-to-end pipeline: perception on the host PC, streamed features over serial, and layer-streamed inference on the Arduino Nano. The same firmware structure hosts any precision or topology variant evaluated in this paper, selected at build time.

Table 2: ATmega328P resource budget

Resource	Total	Reserved	Available to firmware
Flash	32,768 B	2,048 B (bootloader)	30,720 B
SRAM	2,048 B	— (shared: static + stack)	2,048 B
Clock	16 MHz	—	—
FPU	None (software float only)	—	—

3.2. Host-Side Components

A Python 3.12 process captures webcam frames through OpenCV (DirectShow back end on Windows), runs the MediaPipe HandLandmarker (v1.0.0, `hand_landmarker.task`) at a configurable detection-confidence threshold, and passes the raw landmarks to a normalization stage (`features.py`) before transmitting the encoded vector to the microcontroller through `stream.py`.

3.3. Microcontroller-Side Components

The Arduino Nano runs a state-machine serial parser that fills a 240-byte packet buffer, followed by the layer-streamed MLP inference routine described in Section 5. Firmware variants are generated across two independent axes evaluated in this paper; weight precision (float32, int16, int8) and network topology (five sizes, Section 4); all variants share an identical execution strategy, differing only in weight bit-width or layer dimensions. An SSD1306 OLED display can optionally show the predicted letter, with a graceful fallback to serial-only output when no display is attached; Section 7.6 reports the OLED draw call's measured contribution to end-to-end latency separately, since it is not part of on-device inference.

3.4. Communication Protocol

The link operates at 115200 baud. Each outbound packet from the host follows the form `<v1,v2,...,v42,cs>\n`, where `cs` is a checksum over the 42 feature values. The microcontroller replies with `{sign,infer_us,confidence,fps,free_ram}` giving the host a live view of on-device performance without a separate debugging channel, regardless of which precision or topology variant is running.

4. Model Design

4.1. Reference Architecture and Topology Family

The primary evaluation model is a three-layer MLP with topology $42 \rightarrow 32 \rightarrow 16 \rightarrow 5$ (ReLU hidden activations, softmax output), summarized in Table 3. To situate this architecture within the feasible design space rather than treat it as an isolated choice, four additional topologies were trained and deployed identically, spanning roughly 2× smaller to 2.5× larger parameter counts (Table 4); Section 7.2 reports their accuracy, latency, flash, and SRAM as a joint function of scale.

Table 3: Layer-wise neuron and parameter counts (primary topology)

Layer	In	Out	Act.	Weights	Params
Hidden 1	42	32	ReLU	1,344	1,376
Hidden 2	32	16	ReLU	512	528
Output	16	5	Softmax	80	85
Total	42	5	–	**1,936**	**1,989**

Table 4: Candidate MLP topologies

Name	Shape	Parameters	MACs/inference
small1	42→16→8→5	869	840
small2	42→24→12→5	1,397	1,356
current (primary)	**42→32→16→5**	**1,989**	**1,936**
large1	42→48→24→5	3,365	3,288
large2	42→64→32→5	4,997	4,896

4.2. Quantization Scheme

Weights are quantized independently for each output neuron (row-wise quantization of each dense weight matrix), rather than with a single tensor-wide scale: for a weight matrix of shape (out_features, in_features), one scale factor is computed per row, i.e. per output neuron, and applied to every input weight feeding that neuron. For output neuron j, the int16 scale factor is

$$s_j^{16} = \frac{\max_i |w_{ij}|}{2^{15} - 1}, \tag{1}$$

and the int8 scale factor is

$$s_j^{8} = \frac{\max_i |w_{ij}|}{2^{7} - 1}. \tag{2}$$

Each weight is encoded as $w_{ij}^q = \text{clip}(\text{round}(w_{ij}/s_j^q), -2^{b-1}, 2^{b-1} - 1)$ for bit-width $b \in \{16, 8\}$. Accumulation during inference uses 32-bit integers in both cases; for int16, with 42 inputs bounded by 1000 and weights bounded by 32767, the worst-case accumulator magnitude is $42 \times 1000 \times 32767 \approx 1.38 \times 10^9$, safely under the signed 32-bit limit of $2^{31} - 1$. Hidden-layer activations apply ReLU with clamping before being re-quantized for the next layer.

To state this explicitly, the network uses quantized integer weights and integer accumulation throughout the hidden layers, but retains float32 output rescaling and softmax computed via the AVR software floating-point library; the design is therefore **mixed-precision quantized inference**, not fully integer end-to-end. This distinction matters for the streaming-execution discussion in Section 5: quantization reduces weight *storage* size, but does not, by itself, determine whether those weights fit in SRAM that is a separate, execution-strategy question addressed next.

4.3. Memory Footprint

This paper distinguishes three storage figures used consistently by name: **weight storage** (the quantized weight matrices alone), **complete model storage** (weight storage plus biases and, for int16/int8, per-neuron scales), and **complete firmware** (the complete model plus all surrounding code). Table 5 reports the first two; Section 8 reports the third.

Runtime SRAM usage in Table 5 is dominated by the shared int16 activation buffers rather than by the weight tables, because weights are read directly from flash (PROGMEM) during inference rather than copied into SRAM in either the int16 or int8 build. This point is easy to conflate with the layer-streaming execution strategy itself; Section 5 isolates weight-staging as an independent variable and shows that SRAM residency of weights not their quantized size is the binding constraint that determines feasibility.

Reconciling analytical and as-built model-flash figures. Table 5's complete model storage (4,296 B for int16) is the semantically correct footprint: 53 biases actually used by the network (32+16+5 output-dimensioned) at 4 B each,

Table 5: Model memory breakdown: float32 vs. int16 vs. int8

Component	Storage	float32	int16	int8
W0 (32×42)	Flash	5,376 B	2,688 B	1,344 B
W1 (16×32)	Flash	2,048 B	1,024 B	512 B
W2 (5×16)	Flash	320 B	160 B	80 B
Weight storage	**Flash**	**7,744**	**3,872 B**	**1,936 B**
Biases (53 float32)	Flash	212 B	212 B	212 B
Quantization scales	Flash	—	212 B	212 B
Complete model storage	**Flash**	**7,956 B**	**4,296 B**	**2,360 B**

plus 212 B of per-neuron scales. The as-deployed current-topology firmware measured in Tables 10 and 17 (4,444 B) is 148 B larger because the pre-experiments header generator declared each layer's bias array sized to that layer's *input* dimension (`bias0[42]`, `bias1[32]`, `bias2[16]` – 90 floats, 360 B) rather than its output dimension (32+16+5 = 53 floats, 212 B actually used), reserving 148 B of dead, unused flash (3,872 + 212 + 360 = 4,444, exactly). This is an artifact of the exporter version used for the primary-topology deployment, not a measurement error; both 4,296 B and 4,444 B are internally correct for what each reports (semantic requirement vs. as-built binary), and the discrepancy is confined to this specific header. The five-topology scaling firmware family (Section 7.2, generated by the corrected `export_scaling_firmware.py`) declares output-dimensioned bias arrays throughout and carries no such overhead confirmed by `small1`, whose parsed 1,912 B model-flash figure in Table 10 matches its semantic requirement exactly.

5. Layer-Streamed Execution

Section 4 distinguished weight *precision* (how many bits each weight occupies) from weight *staging* (where those weights live during a multiply-accumulate). This section treats staging as an independent experimental variable and asks a question the original submission never isolated: given a trained, quantized model, does copying its weight matrix into SRAM before computing on it provide a net benefit on this device or is it not viable at all?

5.1. Two Execution Strategies

Two strategies were implemented from identical `model.h` headers (same trained checkpoints, same per-neuron row-max int16 quantization), differing only in the weight-read path:

- **Conventional (weights staged in SRAM).** Before computing a layer, its entire weight matrix is copied from flash (PROGMEM) into an SRAM buffer via `memcpy_P()`; every subsequent multiply-accumulate read then hits SRAM directly (approx. 1-cycle-class access) rather than issuing an `LPM` flash read (approx. 3-cycle-class access).
- **Layer-streamed (deployed scheme).** No weight copy occurs; each multiply-accumulate reads one int16 weight directly from PROGMEM via `pgm_read_word()`, accumulating into int32. Weights never occupy SRAM at any point during inference.

Both variants were compiled with `arduino-cli`/`avr-gcc` and, where they compiled, flashed and timed with an identical timing-probe sketch that instruments each layer independently with `micros()` and reports `{sign, infer_us, l0_us, l1_us, l2_us, sm_us, confidence, fps, free_ram, peak_stack}` per packet, median over n = 300 packets with zero timeouts. The comparison spans all five topologies from Table 4, including the primary 1,989-parameter model.

Table 6: Conventional (SRAM-resident) feasibility by topology

Architecture	Static SRAM demanded	% of 2,048 B	Verdict
small1 (869 params)	2,006 B	97.9%	**Compiles** (only 42 B left for locals+stack)
small2 (1,397 params)	2,718 B	132.7%	**Compiler-rejected** (overshoots by 670 B)
current (1,989 params)	3,430 B	167.5%	**Compiler-rejected** (overshoots by 1,382 B)
large1 (3,365 params)	4,854 B	237.0%	**Compiler-rejected** (overshoots by 2,806 B)
large2 (4,997 params)	6,278 B	306.5%	**Compiler-rejected** (overshoots by 4,230 B)

5.2. Conventional Feasibility

Table 6 reports the compiler's static-SRAM verdict for the conventional variant at each topology, against the ATmega328P's 2,048 B budget.

small1 was the only conventional build that compiled, and it was pushed to real hardware despite its 42 B margin; the flash write succeeded (`arduino-cli upload`, old-bootloader Nano), but the board produced zero responses to 20 consecutive packets and never emitted its boot banner it crashed or hung before completing the first inference. The 42 B of static headroom cannot cover even the `loop()`→`parse_packet()`→`predict()` call-chain's local variables, let alone interrupt frames, so this result is reported as **measured infeasible**, not merely marginal.

Every one of the five topologies tested including the paper's primary 1,989-parameter model is therefore infeasible under conventional SRAM-resident weight staging on this device; four are rejected outright by the compiler, and the fifth fails on real silicon despite compiling. There is no conventional baseline left to time.

5.3. Layer-Streamed Reference Measurements

Table 7 reports the deployable side of the comparison, per-layer latency and peak stack depth for the layer-streamed variant at each topology (median of $n = 300$ packets). Peak stack was measured by stack-painting filling SRAM with a sentinel byte (0xCD) at boot and reporting the deepest address overwritten by the time each packet's inference completes. **Inference total, defined precisely.** The `Total` column is the value reported directly by the firmware's `infer_us` timer, which spans Layer 1 through the output layer only; softmax is timed by a separate `sm_us` instrumentation point that closes *after* `infer_us` stops and is therefore not included in `Total`. This is a firmware timing-window choice, not an omission; every latency figure in this paper labeled "inference latency," "Total," or used in the Pareto and end-to-end analyses (Sections 7.7, 7.6) refers to this same `infer_us`-defined, softmax-excluded quantity, applied consistently across all seven configurations.

Table 7: Layer-streamed timing and peak stack by topology

Architecture	Layer 1 (μs)	Layer 2 (μs)	Output (μs)	Softmax (μs, excl. from Total)	Total (μs) = `infer_us`	Peak stack
small1	3,632	892	352	1,752	4,880	97
small2	5,460	1,768	444	2,376	7,672	97
current (int16)	7,208	2,904	536	984	10,656	97-101
large1	10,864	6,108	708	3,956	17,680	97
large2	14,476	10,344	892	6,396	25,712	97

*Peak stack was stack-painted at 97 B (int8) and 101 B (float32) on the current topology (n=300 each) and at 97 B on all four scaled topologies; the int16 current build shares the identical call chain and frame layout with its int8/float32 siblings (only the MAC element width differs, with 4-byte accumulators in all three), so the same 97–101 B band is reported here as the applicable range rather than a single measured value.

5.4. Bounding the Streaming Overhead Analytically

Although no conventional build could be timed beyond small1's crash, the ablation question of what layer-streaming costs, if anything admits a simple analytical upper bound even without that data. The only mechanism by which a conventional, SRAM-resident layout could outperform streaming is replacing each `LPM` flash read (approx. 3-cycle class) with an SRAM read (approx. 1-cycle class) at most 2 cycles saved per weight access. A full

multiply-accumulate on the AVR (operand load, 16-bit multiply, 32-bit accumulate) costs on the order of 8–9 cycles in total, of which the weight read is only one component; on that basis, the 2-cycle read saving corresponds to roughly 22–25% of one MAC's cycle cost, giving an approximate upper bound on conventional staging's best-case speedup well under 25% of total multiply-accumulate time. This bound is order-of-magnitude, built on approximate AVR cycle-class figures for LPM and SRAM access rather than a cycle-accurate simulation, and is intended to show that no plausible SRAM-residency benefit could approach the cost of the SRAM it would consume (Table 6), not to predict a precise conventional-build latency.

Against that bounded, unrealized benefit, the cost is concrete and already measured $N_{\text{weights}} \times 2$ bytes of SRAM, ranging from 1,344 B (small1) to 6,400 B (large2) across the topologies in this study a cost that Table 6 shows the 2,048 B part cannot pay at *any* tested size once activations, the serial buffer, and a live call stack are also accounted for. Layer-streaming is therefore not offered as the faster of two competitive strategies; within the topology range tested here, it is the only weight-staging strategy that executes at all on this hardware, and its per-layer timings in Table 7 already include whatever LPM-read penalty a hypothetical SRAM-resident version would have avoided.

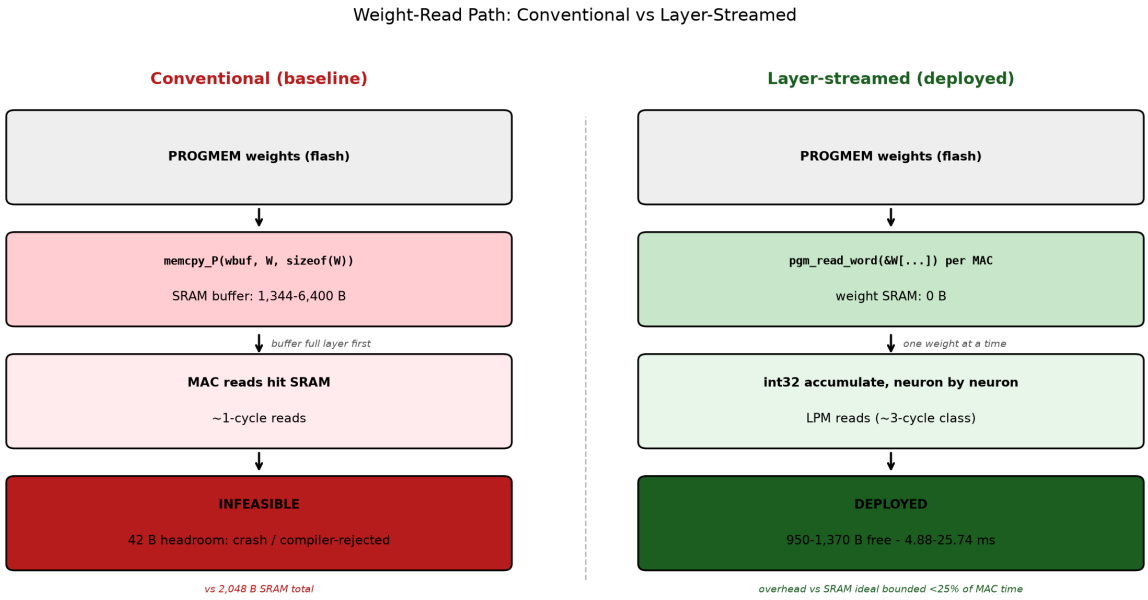

Figure 2: Weight-read path comparison: conventional staging (`memcpy_P` into a 1,344–6,400 B SRAM buffer, then ~1-cycle SRAM reads) versus layer-streamed execution (direct `pgm_read_word` per multiply-accumulate, 0 B weight SRAM, ~3-cycle LPM reads). Conventional staging is infeasible at every tested topology size, crashing or failing to compile even where its theoretical SRAM margin appeared positive; layer-streaming deploys successfully across all five topologies at 950–1,370 B free SRAM and 4.88–25.71 ms latency.

6. Training and Evaluation Methodology

6.1. Toolchain and Build Configuration

All firmware variants were compiled with `arduino-cli` targeting `arduino:avr:nano:cpu=atmega328old` (an old-bootloader Nano) and flashed over USB-serial. Compiler-reported static-SRAM and flash figures are taken directly from `arduino-cli compile` output rather than estimated; no cycle-accurate simulation or memory-usage model is used anywhere in this paper. Default `avr-gcc` optimization settings shipped with the Arduino AVR core were used unmodified across all variants.

6.2. Dataset and Split

Training examples were collected from a webcam using `collect_data.py` for five ASL letters (A–E), yielding roughly 2,000 samples per class and 10,037 samples overall from a single subject across multiple recording sessions

spanning different backgrounds, lighting conditions, and hand orientations. The data were split 70/15/15 into training (7,025), validation (1,505), and test (1,507) partitions. This split is randomized across the combined recording pool rather than held out by session or subject; Section 10 discusses what this does and does not establish about generalization.

6.3. Training Configuration

The floating-point model was trained in PyTorch 2.13.0+cpu on Python 3.12 (training and all Python-side analysis scripts ran under this same interpreter; using Adam (lr = 1×10^{-3}), cross-entropy loss, batch size 32, 100 epochs, Kaiming-uniform initialization [2]. All five topologies in Table 4 were trained under this identical procedure from independent random initializations; int16 and int8 variants of each topology are derived from the same trained float32 checkpoint by post-training quantization, so any accuracy difference between precisions at a given topology reflects the quantization step itself rather than differences in training.

6.4. Multi-Seed Topology Replication Protocol

The original topology sweep (Section 7.2) trained each of the five architectures in Table 4 from a single random initialization, a limitation that leaves open whether the resulting accuracy ranking and specifically the primary current topology's comparatively low single-run accuracy relative to small2 reflects a genuine architectural effect or an artifact of initialization. To resolve this, each of the five topologies was retrained from five independent random seeds ($\{42, 0, 1, 2, 123\}$) using the identical procedure described in Section 6.3 (same optimizer, learning rate, batch size, epoch count, and train/validation/test split), yielding 25 total training runs. Test accuracy for every run was computed on the same fixed 1,507-sample held-out test partition (Section 6.2), so all 25 accuracies are directly comparable.

One checkpoint per topology the same checkpoint whose quantized export was flashed to hardware and used for every latency, memory, per-class, confusion-matrix, PCA, and live-confidence measurement reported elsewhere in this paper (Sections 7.1–7.6) was retained as the **frozen deployed checkpoint** for that topology. This checkpoint is reported alongside, but is not conflated with, the five-seed statistics; it establishes what was physically measured on hardware, while the five-seed mean and standard deviation characterize the topology's accuracy as a training procedure rather than as one specific trained instance. For small1, small2, large1, and large2, the frozen deployed checkpoint corresponds exactly to one of the five fresh seeds (small1: seed 123; small2, large1, large2: seed 42), so its accuracy is already included in that topology's five-seed statistics. **The primary topology's deployed checkpoint is the exception**; it is a pre-existing artifact (`model.pt`) trained under an earlier iteration of the export pipeline before the five-seed replication protocol was introduced, and is not one of the five fresh seeds reported for current in Table 11, its accuracy (89.58%) in fact falls below all five fresh current-topology seeds (90.58–94.49%). It is retained, unretrained, as the frozen deployed checkpoint specifically because it is the exact binary underlying every current-topology hardware measurement elsewhere in this paper; discarding it in favor of a higher-accuracy fresh seed would invalidate the internal consistency of those measurements. This discrepancy is discussed further in Section 7.2.

6.5. On-Device Timing Protocol

All latency figures are the median of $n = 300$ inference packets per configuration, collected with the timing-probe firmware described in Section 5, which instruments each of the four inference stages (Layer 1, Layer 2, output layer, softmax) independently with `micros()` calls. All timing runs recorded zero serial timeouts. **Definition of "inference latency."** The firmware's `infer_us` timer the quantity reported everywhere in this paper as "inference latency," "Total," or as the latency figure in Tables 7, 12, 10, and 17 spans dense-layer computation only (Layer 1, Layer 2, output rescaling); it closes before softmax begins and does not include it. Softmax is timed separately (`sm_us`) and is reported alongside `infer_us` wherever both are available, but is not summed into it. This definition is applied uniformly across all three precisions and all five topologies.

6.6. Confidence-Logging Protocol

Two distinct confidence datasets are used in this paper and are not interchangeable: (i) **test-set confidence**, computed offline from the trained model's softmax output on all 1,507 held-out test samples per precision (Section 7.1); and (ii) **live streaming confidence**, logged from the on-device softmax output during normal streaming operation over independent sessions of approximately 1,300–1,500 packets per precision (Section 7.4). The live sessions are not

drawn from the fixed test-set sample pool and were not synchronized in length across precisions (int16: n = 1,514; float32: n = 1,314; int8: n = 1,364), so live-confidence comparisons report distributional statistics rather than paired per-sample comparisons.

6.7. End-to-End Latency Measurement

The pipeline-stage breakdown in Section 7.6 was measured with the host-side `-live` instrumentation flag in the capture/streaming script, which timestamps each stage boundary independently; MCU-inference time is taken from the same on-device median reported in Section 5. OLED draw time was measured separately (n = 300) using the SSD1306Ascii text-mode driver on an SH1106 128×64 device; the Adafruit framebuffer-based driver was not used because its SRAM footprint does not fit alongside the model's activation buffers on this device.

7. Results and Discussion

7.1. Precision Comparison: Accuracy, Confusion, and Agreement

Table 8 gives the aggregate view across all three precisions on the primary topology, recomputed independently from raw per-sample predictions rather than assumed identical from a single confusion matrix. Int16 is indistinguishable from the floating-point baseline, identical accuracy and 100% argmax agreement no test-set letter is reclassified as a result of int16 quantization. Int8 trails by roughly one accuracy point.

Table 8: Overall metrics: float32 vs. int16 vs. int8 (n = 1,507 test samples)

Metric	Float32	int16	int8
Accuracy	0.8958	0.8958	0.8845
Argmax agreement (vs. float32)	—	1.0000	0.9801

Table 9 reports per-class precision/recall/F1, computed separately for each precision from its own confusion matrix rather than assumed shared between float32 and int16, the recomputation confirms they are in fact cell-for-cell identical for these two precisions, a *result* of int16's 100% argmax agreement rather than an assumption carried into the analysis.

Table 9: Per-class metrics by precision

	Float32			int16			int8			
Class	P	R	F1	P	R	F1	P	R	F1	Support
A	0.927	0.737	0.821	0.927	0.737	0.821	0.923	0.692	0.791	312
B	0.904	0.868	0.886	0.904	0.868	0.886	0.911	0.872	0.891	304
C	0.749	0.978	0.848	0.749	0.978	0.848	0.714	0.964	0.820	277
D	0.942	0.968	0.955	0.942	0.968	0.955	0.933	0.972	0.952	316
E	0.996	0.936	0.965	0.996	0.936	0.965	0.996	0.933	0.964	298

The dominant error mode across all three precisions is A misclassified as C: 59 of 312 A samples under float32/int16, rising to 76 under int8. This is examined further in Section 7.5. These figures are reported as within-subject, A–E test accuracy under the stated random split (Section 6.2), not as general ASL recognition accuracy.

7.2. Topology Scaling

Table 10 reports each topology's accuracy, latency, and flash/SRAM footprint, showing how the design space moves as a function of scale. Following the single-seed limitation flagged in the original submission, accuracy is now reported primarily as the five-seed mean ± standard deviation established in Section 6.4 (n = 5 independent training runs per topology) to characterize training variability across independent initializations, with the frozen hardware-deployed checkpoint's accuracy given as a secondary, explicitly labeled column; latency, model flash, and static SRAM are architectural properties of the frozen deployed checkpoint and are not seed-dependent.

Table 10: Topology scaling: accuracy (5-seed mean ± SD, primary statistic), deployed checkpoint (secondary), latency, and footprint

Architecture	Params	Accuracy, 5-seed mean ± SD	Deployed checkpoint	Latency (μs)	Model flash (B)	Static SRAM (B)
small1 (42→16→8→5)	869	62.54% ± 40.28	92.70%†	4,880	1,912	678
small2 (42→24→12→5)	1,397	78.83% ± 33.80	94.69%†	7,672	3,040	718
current (42→32→16→5)	**1,989**	**92.46% ± 1.85**	**89.58%**‡	**10,656**	**4,444**	**1,098**
large1 (42→48→24→5)	3,365	93.71% ± 0.61	93.56%†	17,680	7,192	838
large2 (42→64→32→5)	4,997	94.08% ± 0.38	94.03%†	25,712	10,600	918

†The deployed checkpoint is one of the five fresh seeds and is already included in that topology's mean. ‡The primary topology's deployed checkpoint is a separate, pre-existing checkpoint (`model.pt`) trained before the five-seed protocol was introduced (Section 6.4) and is *not* included in current's five-seed mean; it underperforms all five fresh current-topology seeds.

The five-seed mean is dramatically lower for small1 (62.54% vs. 92.70%) and small2 (78.83% vs. 94.69%), for a specific and diagnosable reason both narrower topologies are prone to a catastrophic training-failure mode in which a run converges to near-chance accuracy (18.4% for a five-way classification problem, against a 20% random baseline) rather than to a usable classifier. Table 11 isolates this failure mode explicitly. small1 collapsed in 2 of 5 seeds and small2 in 1 of 5; current, large1, and large2 all wider at the first hidden layer collapsed in 0 of 5 seeds each. It indicates that narrow first-hidden-layer width (16 or 24 units, versus 32–64 for current/large1/large2) leaves this network family vulnerable to a bad initialization from which training does not recover under the fixed optimizer and learning-rate schedule used throughout this paper (Section 6.3), rather than merely producing higher run-to-run variance around a stable optimum.

Table 11: Seed-fragility analysis: collapse incidence and converged-only accuracy (n = 5 seeds per topology)

Architecture	Runs collapsed (of 5)	Collapsed-run accuracy range	Converged-only mean ± SD (n excl. collapses)
small1	2	18.38–18.45%	91.95% ± 1.20 (n = 3)
small2	1	18.38%	93.95% ± 0.90 (n = 4)
current	0	—	92.46% ± 1.85 (n = 5)
large1	0	—	93.71% ± 0.61 (n = 5)
large2	0	—	94.08% ± 0.38 (n = 5)

On the collapse-inclusive five-seed mean, the statistic that reflects what a practitioner should expect from training this recipe once current (92.46% ± 1.85, 0/5 collapsed) exceeds small2 (78.83% ± 33.80, 1/5 collapsed) and small1 (62.54% ± 40.28, 2/5 collapsed). small2's single deployed run (94.69%) exceeded current's single deployed run (89.58%) was, in retrospect, a comparison of asmall2 seed against (and, as Section 6.4 notes, procedurally distinct and pre-dated) current checkpoint, not a stable architectural effect excluding small2's collapsed run, its *converged-only* mean (93.95% ± 0.90) is still numerically higher than current's mean by about 1.5 points, but this gap is well within current's own ±1.85-point run-to-run spread and is not read as a reliable advantage given small2's demonstrated 20% collapse rate, which current does not share. **large1 and large2 present a more credible case for outperforming current** both exceed current's mean accuracy by 1.2–1.6 points while simultaneously having much tighter standard deviations (0.61 and 0.38 points, versus current's 1.85) and zero observed collapses across their five seeds.

Latency and flash scale monotonically and linearly with parameter count, as expected, since weight reads dominate both, and these architectural properties are unaffected by the seed-fragility finding above. Fig. 3 plots the five-seed mean accuracy (with error bars) against parameter count alongside latency, visualizing both the flat-to-rising accuracy trend for the three collapse-free topologies and the much wider uncertainty bars for small1 and small2.

The more decisive result of this sweep concerns feasibility rather than accuracy, static SRAM usage under layer-streaming stays well under budget at every size tested (678–1,098 B of 2,048 B), including large2, the largest topology, at 4,997 parameters which remains comfortably feasible (918 B, 45% of budget) despite being 2.5× larger than the primary current model, which Table 6 showed was compiler-rejected under conventional staging. This confirms that within the size range tested, weight-staging strategy not raw model size is the binding feasibility constraint on this device, and this conclusion is entirely unaffected by the seed-fragility finding, which concerns accuracy and training

reliability rather than memory footprint.

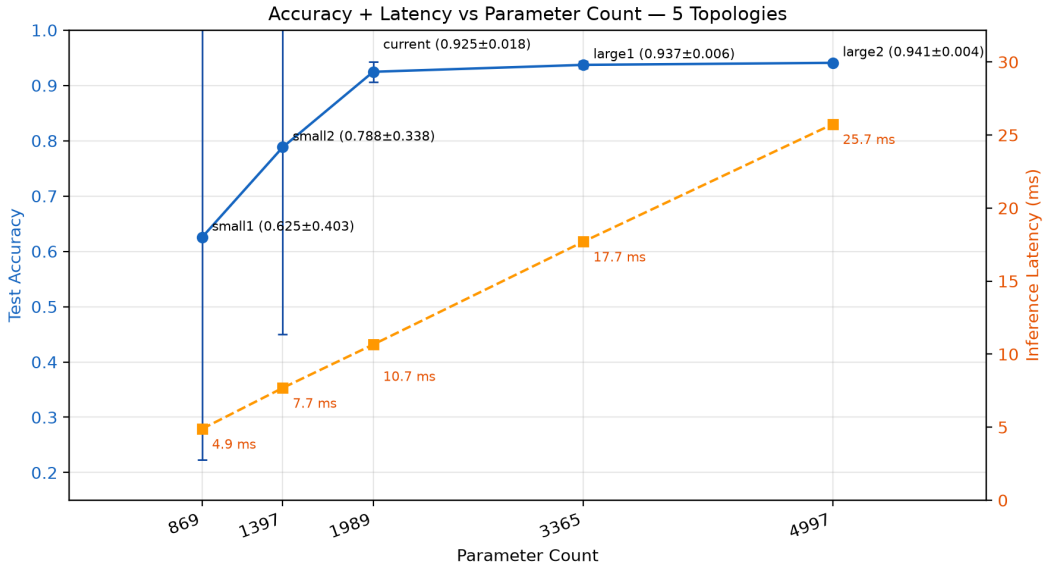

Figure 3: Five-seed mean test accuracy with ±1 SD error bars (left axis) and frozen-deployed-checkpoint inference latency (right axis) versus parameter count across the five evaluated topologies. The wide error bars on small1 (0.625 ± 0.403) and small2 (0.788 ± 0.338) reflect their catastrophic-collapse failure mode (Table 11), not ordinary training noise; current (0.925 ± 0.018), large1 (0.937 ± 0.006), and large2 (0.941 ± 0.004) show tight, collapse-free distributions. Latency scales monotonically with parameter count (4.9–25.7 ms) and is unaffected by the accuracy-side seed fragility.

7.3. Layer-Level Latency

Table 12 decomposes on-device latency into its four instrumented stages for the primary topology at all three precisions. As established in Section 5, `Total` is the firmware's `infer_us` timer (Layer 1 + Layer 2 + output) and does not include softmax, which is timed separately by `sm_us` after `infer_us` has already closed; the Softmax row is shown here for completeness and comparison across precisions, not as a component that sums into `Total`.

Table 12: Layer-level latency by precision (primary topology, median of $n = 300$)

Stage	Float32 (μs)	int16 (μs)	int8 (μs)
Layer 1	39,072	7,208	6,940
Layer 2	9,908	2,904	2,788
Output	1,612	536	516
Total (= `infer_us`)	**50,588**	**10,656**	**10,240**
Softmax (μs, excl. from Total)	2,712	984	2,692

The dense-layer stages (Layer 1, Layer 2, output) behave as expected float32→int16 collapses each stage by roughly 5–6×, since the ATmega328P's software floating-point multiply-accumulate is replaced by 32-bit integer accumulation implemented via the AVR's native integer instruction set, and int16→int8 yields a further small reduction (2–4% per stage) consistent with smaller per-weight PROGMEM reads.

Softmax does not follow this pattern int8's softmax stage (2,692 μs) is close to float32's (2,712 μs) and nearly 2.7× slower than int16's (984 μs). Softmax is computed identically in all three firmware variants via the AVR software floating-point exponential, so this is not an int8-specific code path. A plausible explanation is that AVR's software `exp()` has data-dependent execution time, and int8's coarser quantization shifts the five raw output scores into a

numeric range that happens to be more expensive for this routine, coincidentally closer to float32's range than to int16's. This is reported as an empirical timing observation with a plausible mechanism (Section 10), not an isolated and confirmed cause.

This finding matters for how int8's speed advantage should be scoped. Smaller per-weight PROGMEM reads give int8 a real, consistent 416 μs advantage over int16 in `infer_us` (10,656 vs. 10,240 μs) and because `infer_us` is what this paper reports throughout as "inference latency" (Table 17, Section 7.6), that 416 μs advantage is real and holds for every headline latency figure in this paper. It is not, however, a general-purpose latency advantage if a downstream design must wait for softmax to complete before acting on a prediction i.e., if the relevant latency budget is `infer_us` + `sm_us` int8 is *slower* than int16 by roughly 1.3 ms (12,932 vs. 11,640 μs combined), because int8's anomalous softmax cost more than reverses its dense-layer saving. Attributing int8's speed advantage to memory-read size alone, without this scope caveat, would overstate a benefit that depends entirely on whether softmax sits inside or outside the latency budget being optimized.

Fig. 4 places this decomposition in the context of every variant evaluated in this paper all five topologies plus the primary topology's float32 and int8 builds, and a build with the OLED display attached (`current+OLED`) included to check whether driving the display perturbs the inference stages themselves. **The bar totals in this figure are dense + softmax combined** (the sum of all four stacked segments), which is a different quantity from the `infer_us`-defined "Total" reported in Tables 7, 12, 10, and 17 and used throughout the rest of this paper (Section 5); for example, the primary topology's `infer_us` total is 10.66 ms, while its dense+softmax total shown here is 11.63 ms (10,656 + 984 μs $\approx$ 7,208 + 2,904 + 536 + 984 μs). The two are reconciled explicitly in Section 7.3; `current+OLED`'s stage breakdown and dense+softmax total (11.63 ms) closely match the plain `current` (int16) build, confirming that the OLED's cost, measured separately as 65.86 ms in Section 7.6, is incurred entirely in the display draw call after inference completes rather than by contending for CPU time during the inference stages themselves. Across topologies, Layer 1 is the dominant cost at every scale, consistent with it containing the largest weight matrix (Table 4), and float32's Layer 1 stage alone (39.1 ms) exceeds every quantized build's dense+softmax total shown here.

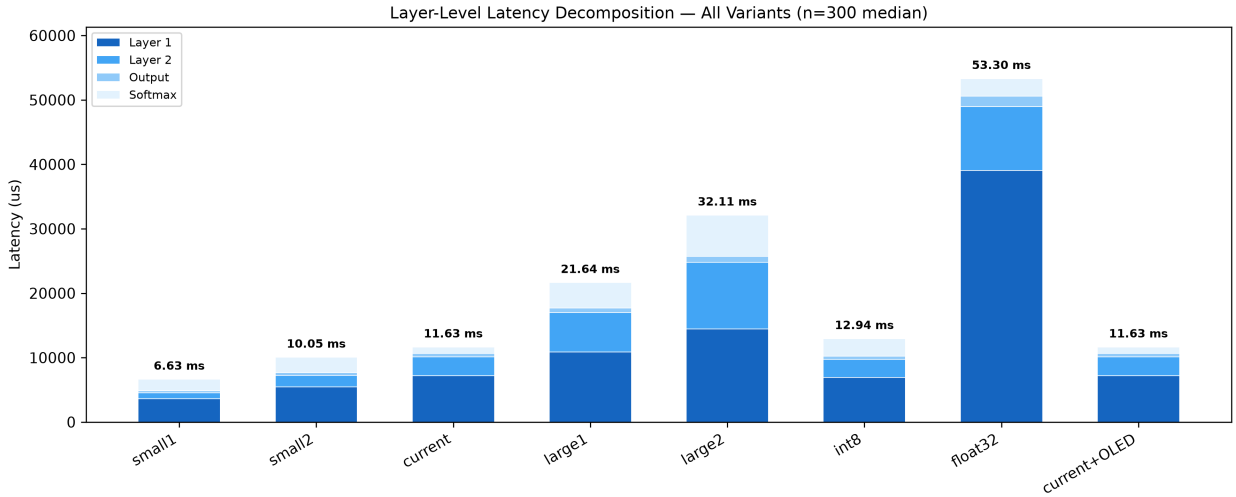

Figure 4: Layer-level latency decomposition (median of $n = 300$) for all eight evaluated firmware variants the five int16 topologies, the primary topology's int8 and float32 builds, and an int16 build with the OLED display attached (`current+OLED`). The labeled total above each bar is the sum of all four stacked segments (Layer 1 + Layer 2 + Output + Softmax) a dense+softmax combined figure, distinct from the `infer_us`-defined, softmax-excluded "Total" used elsewhere in this paper (Section 5; e.g. 11.63 ms here vs. 10.66 ms `infer_us` for `current`). Layer 1 dominates total latency at every scale; `current+OLED`'s stage breakdown and total match plain `current` almost exactly, confirming the OLED's latency cost (Section 7.6) is incurred after inference completes rather than during it.

7.4. Confidence and Reliability

Test-set accuracy and argmax agreement (Section 7.1) describe how often each precision is *correct*;

Table 13: Live streaming confidence by precision (independent sessions)

Source	n	Mean confidence	Std. dev.
Float32 (live)	1,314	92.88%	12.96
int16 (live)	1,514	93.91%	12.17
int8 (live)	1,364	90.23%	15.00

Because these sessions were run independently and are not equal-length or paired sample-for-sample (Section 6), they support a distributional comparison rather than a per-sample one. Read that way, int16 produced a confidence distribution closely matching float32's under these independent live-streaming sessions, its mean confidence (93.91%) and spread (12.17 std. dev.) sit close to float32's (92.88%, 12.96), with int16 marginally higher and tighter, consistent with ordinary run-to-run variation rather than a systematic effect. Int8 produced both a lower mean confidence and a wider confidence distribution than int16, its mean (90.23%) trails int16 by 3.68 points and its spread (15.00) is visibly wider a divergence on confidence larger, proportionally, than int8's divergence on accuracy.

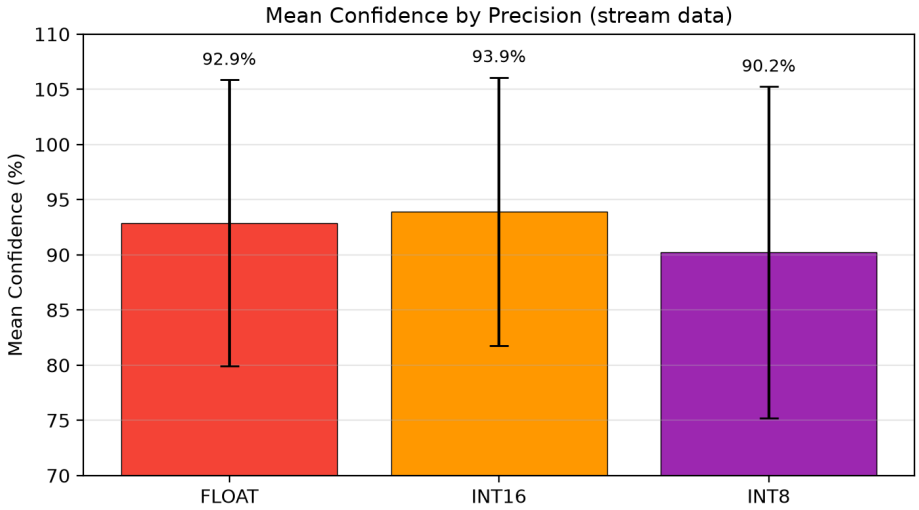

Figure 5: Mean live streaming confidence by precision, with error bars showing one standard deviation (n = 1,314 float32; n = 1,514 int16; n = 1,364 int8; independent, unpaired sessions). Int16's mean sits marginally above float32's with a comparable spread, consistent with a closely matching confidence distribution; int8's mean sits below both, and its wider error bar reflects the less consistent confidence distribution noted in the text.

This bucket-level reliability analysis reports whether a stated confidence level tracks the empirical probability of correctness, rather than only whether confidence is high; it does not compute expected calibration error, Brier score, or a formal reliability diagram, and the claims below are scoped to what the bucketed accuracy figures directly show. The 95–100% bucket, which holds the large majority of test samples under both groupings (1,145 of 1,507 for float32/int16; 1,151 of 1,507 for int8), achieves 98.8% (float32/int16) and 98.3% (int8) empirical accuracy. Both groupings show substantial overconfidence in the mid-range; the 40–60% and 60–80% buckets sit at roughly 51–56% accuracy for both barely above chance for a 5-way classification problem despite reporting 40–80% confidence a pattern consistent with the calibration gap documented broadly in modern neural networks [14]. Int8's bucket accuracies run consistently a few points below float32/int16's at every confidence level except the extremes, which is consistent with its overall accuracy loss (Section 7.1) rather than a qualitatively different reliability pattern. This has a direct design implication for confidence-threshold gating, a system using a ≥95% confidence threshold is well supported by the 95–100% bucket under either precision grouping (98.8%/98.3% empirical accuracy); by contrast, the 90–95% bucket which a ≥90% threshold would also admit achieves only 73.9% (float32/int16) or 70.0% (int8), so a ≥90% cutoff would silently mix a well-supported band with a substantially weaker one under any precision. Anything

Table 14: Confidence-bucket reliability analysis, test set (n = 1,507): float32/int16 (cell-identical, consistent with their 100% argmax agreement) vs. int8

Confidence bucket	Float32/int16		int8	
	n	Accuracy within bucket	n	Accuracy within bucket
0–20%	0	—	0	—
20–40%	12	58.3%	23	47.8%
40–60%	118	53.4%	113	51.3%
60–80%	112	56.3%	108	53.7%
80–90%	74	70.3%	72	63.9%
90–95%	46	73.9%	40	70.0%
95–100%	1,145	98.8%	1,151	98.3%

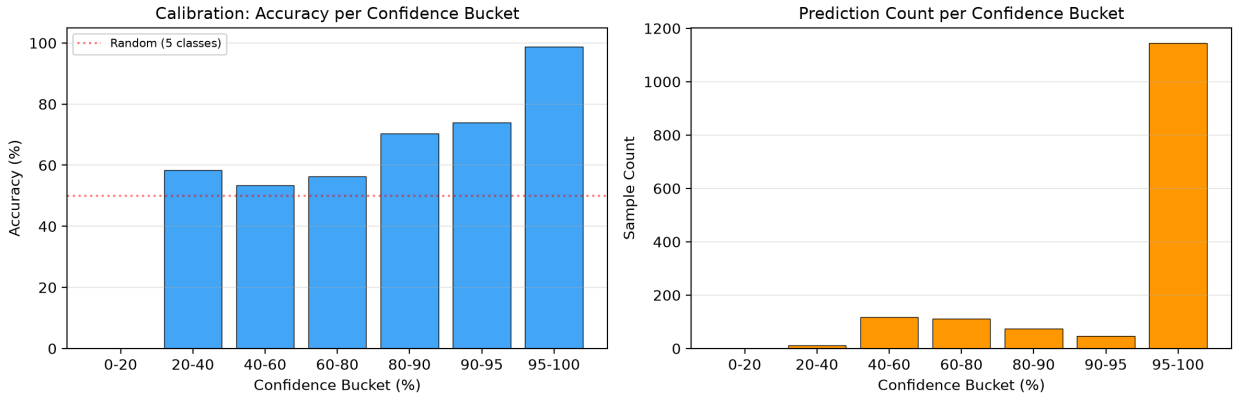

Figure 6: Confidence-bucket reliability analysis on the test set (n = 1,507). Left: empirical accuracy within each confidence bucket against the 20% chance baseline for five-way classification (dotted line), shown for float32/int16; accuracy climbs from 53–58% in the 20–80% buckets to 98.8% in the 95–100% bucket. Right: sample count per bucket for float32/int16, showing the large majority of predictions (1,145 of 1,507) fall in the 95–100% bucket. Int8's corresponding bucket accuracies (Table 14) follow the same overconfidence pattern, consistently a few points lower at every confidence level except the extremes.

in the 40–80% range should be treated as a rejection signal rather than a low-confidence-but-still-usable answer.

7.5. Confusion Analysis

Table 9 already showed that classes A and C drive the accuracy gap between int8 and the other two precisions. Table 15 reports the confusion matrices directly.

The dominant error mode at every precision is A predicted as C: 59 of 312 A samples under float32/int16, rising to 76 under int8. Class A's correct-prediction count correspondingly falls from 230 (float32/int16) to 216 (int8). This is the single largest driver of int8's overall accuracy gap.

7.5.1. Representation-Space Analysis

To examine why the A→C confusion persists across all three precisions rather than appearing only as a quantization artifact, Fig. 8 projects hidden-layer activations for all five test-set classes onto their top two principal components, separately for hidden layer 1 (32-dimensional) and hidden layer 2 (16-dimensional). The two panels show the network progressively sharpening class separation with depth classes B, D, and E, which are cleanly separated in the confusion matrices of Table 15, become visibly tighter and more distinctly separated moving from hidden layer 1 to hidden layer 2.

Class A does not follow this pattern. In both hidden layers, A splits into two sub-clusters rather than forming a single compact group, and in hidden layer 2 the representation immediately feeding the output layer one of these sub-clusters sits directly inside C's own compact cluster rather than occupying separate territory. This is the representation-

Table 15: Confusion matrices by precision (test set, n = 1,507)

Float32 / int16 (cell-for-cell identical)					
True \ Pred	A	B	C	D	E
A	230	17	59	6	0
B	7	264	25	8	0
C	0	3	271	3	0
D	0	5	4	306	1
E	11	3	3	2	279
int8					
True \ Pred	A	B	C	D	E
A	216	13	76	7	0
B	6	265	25	8	0
C	0	5	267	5	0
D	0	5	3	307	1
E	12	3	3	2	278

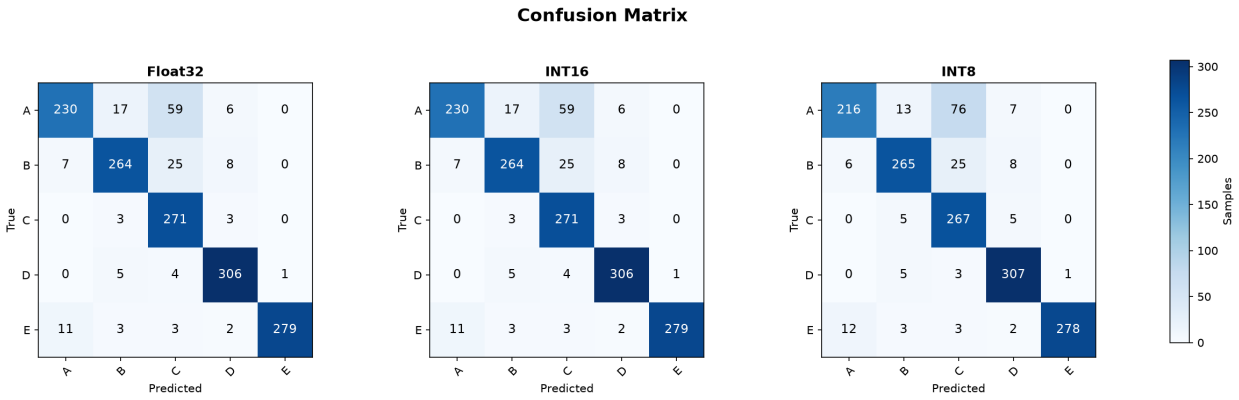

Figure 7: Confusion matrices for all three precisions on the 1,507-sample test set. Float32 and int16 (left, middle) are cell-for-cell identical, each with 59 of 312 A samples misclassified as C. Under int8 (right), this A→C confusion grows to 76 samples and A's correct-prediction count drops from 230 to 216, while classes D and E remain the cleanest under all three precisions.

space signature of the confusion matrix's dominant error mode; at this level of the learned representation, and at every weight precision, there is no visible separation between this A sub-cluster and C's cluster. This suggests the dominant A→C error is associated with overlapping learned representations rather than being introduced solely by numerical quantization; the PCA projection shows representational overlap, but does not by itself establish that hand-pose similarity in the original landmark space is the underlying cause, which would require a separate analysis of the raw landmarks. B and D, by contrast, occupy well-separated regions in hidden layer 2, consistent with their comparatively clean confusion-matrix rows in Table 15.

This localizes the dominant error mode to the feature representation, upstream of the classifier and of the numeric precision used at inference, though the specific role of hand-pose similarity between certain A and C executions remains an inference from the representation-space evidence rather than a directly tested claim. Int8's coarser quantization does not create this confusion; it widens an error mode that already exists in the floating-point model's decision boundary, since the same A/C overlap is visible in the hidden-layer activations regardless of which precision produced them.

7.6. Communication and End-to-End Latency Breakdown

Section 7.3 isolated on-device inference latency down to the individual layer. This section places that number in context by decomposing the entire pipeline into its constituent stages.

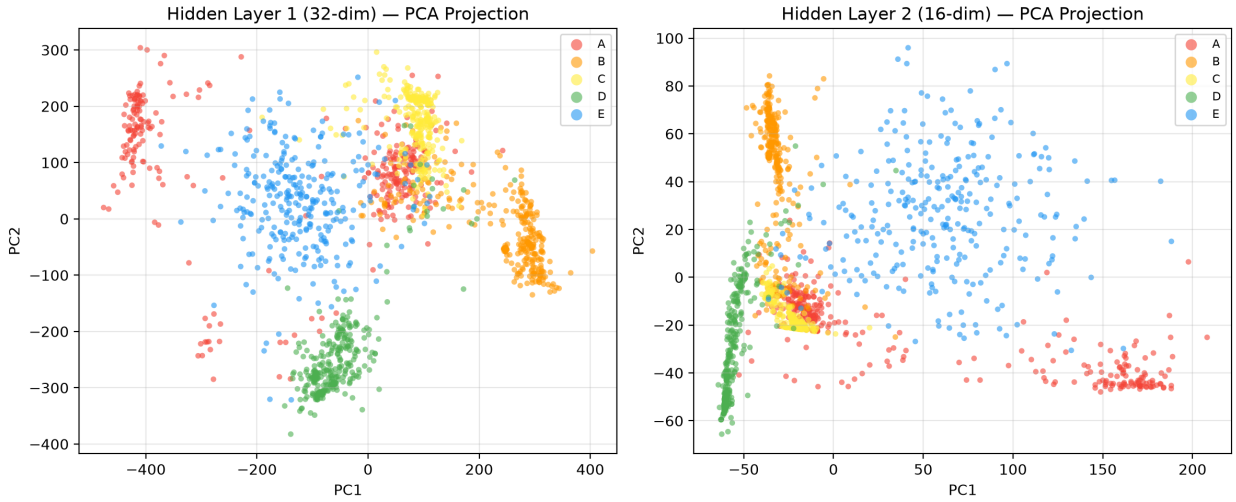

Figure 8: PCA projection of hidden-layer activations for all five test-set classes. Left: hidden layer 1 (32-dim); right: hidden layer 2 (16-dim). Class separation sharpens substantially from layer 1 to layer 2 for B, D, and E, but class A remains split across two sub-clusters in both layers, with one A sub-cluster overlapping C's compact cluster directly, the representation-space signature of the A→C confusion in Fig. 7.

Table 16: End-to-end pipeline latency breakdown

Stage	Time	Measurement basis
Camera capture	32.677 ms	Measured, `-live` instrumentation
MediaPipe (host)	24.285 ms	Measured, `-live` instrumentation
Feature processing	0.173 ms	Measured, `-live` instrumentation
Residual communication/host overhead	19.400 ms	Derived: median(latency_pc_ms) − median(infer_us)
MCU inference (int16)	**10.700 ms**	Median `infer_us`, streaming log
OLED draw (optional)	65.86 ms	Measured, n = 300, SH1106 128×64

The measurements show that host-side perception (camera capture + MediaPipe), at roughly 57 ms combined, is the dominant term in total system latency by a wide margin more than 5× the entire on-device inference cost. On-device classification, the subject of every measurement in Sections 5–7.5 and the paper's primary contribution, contributes only about 10.7 ms to a pipeline whose non-OLED total runs to roughly 87 ms. The residual communication/host overhead is approximately 19.4 ms, nearly twice the MCU inference time it is measured against; because this figure is derived as a difference of medians (Table 16) rather than from independently timestamped serial-transmission and reception events, it is reported as an aggregate residual rather than a directly measured serial round-trip time.

The 4.7× latency reduction from float32 to int16 quantization (Section 7.3) is a substantial and real engineering result for the classification stage specifically with int16 inference occupying approximately 10.7 ms of the approximately 87 ms non-OLED pipeline, optimizing the MCU classifier addresses only about 12% of the deployed pipeline's latency budget. The OLED draw call is reported separately and excluded from the "total" figure above because it is optional output rather than part of the classification pipeline; at 65.86 ms, it would in fact dominate total latency if enabled.

7.7. Pareto Analysis

The preceding subsections examined precision (Section 7.1) and topology (Section 7.2) as separate axes. Table 17 combines both into a single seven-configuration comparison, five int16 topologies plus float32 and int8 at the primary topology. **Accuracy and hardware-measured columns in this table are two different kinds of quantity and are labeled accordingly.** For the five int16 topology rows, accuracy is the five-seed mean established in Section 7.2 (Table 10), since that is the statistically defensible representation of each architecture's achievable accuracy; latency, flash, and static SRAM in every row, by contrast, are properties measured on the single frozen deployed checkpoint for

that configuration (Section 6.4) and are not averaged across seeds a practitioner deploying a different one of the five seeds for a given topology would obtain materially different accuracy for small1/small2 (Table 11) but architecturally identical latency, flash, and SRAM figures, since those depend only on network shape and precision, not on the specific learned weights. The current int8 and current float32 rows use the single deployed checkpoint's accuracy directly, since precision quantization was only swept for the deployed checkpoint and was not repeated across the five fresh seeds; their accuracy is therefore not directly comparable in kind to the mean-based topology rows, only in scale. **Model flash is a subset of firmware flash, not an addition to it**; the model-flash column reports how much of the total compiled firmware is attributable to the weight/bias/scale arrays specifically, so total on-device flash consumption for a configuration is the firmware-flash figure alone, never firmware + model. "Firmware flash" values are as-built, header-parsed figures rather than analytical sums; for the current topology's model-flash figure this includes the 148 B dead-bias overhead discussed in Section 4.3 (4,444 B as-built vs. 4,296 B semantically required), which does not affect the firmware-flash total.

Table 17: Combined precision/topology comparison ($n = 7$ configurations). Accuracy for int16 topology rows is the 5-seed mean (Table 10); accuracy for int8/float32 rows is the single deployed checkpoint's accuracy. All latency/flash/SRAM/confidence figures are deployed-checkpoint hardware measurements throughout.

Configuration	Accuracy	Latency (ms)	Firmware flash (B)	Model flash (B, subset of firmware)	Static SRAM (B)	Mean confidence
small1 (int16)	62.54%*	4.88	8,334	1,912	678	—
small2 (int16)	78.83%*	7.67	9,464	3,040	718	—
current — int16	**92.46%**†	**10.66**	**23,778**	**4,444**	**1,098**	**93.91%**
current — int8	88.45%‡	10.24	21,884	2,360	1,118	90.23%
large1 (int16)	93.71%†	17.68	13,616	7,192	838	—
large2 (int16)	94.08%†	25.71	17,024	10,600	918	—
current — float32	89.58%‡	50.59	27,326	7,956	1,222	92.88%

*5-seed mean, collapse-inclusive (Table 11); converged-only means are 91.95% (small1, $n = 3$) and 93.95% (small2, $n = 4$). †5-seed mean, zero collapses observed. ‡Single deployed-checkpoint accuracy, not a multi-seed statistic.

Three Pareto-relevant observations follow, updated in light of the five-seed replication (Section 6.4). **Float32 is dominated outright**, current float32 matches current's deployed-checkpoint int16 accuracy exactly but loses on every other axis (4.7× the latency, more firmware and model flash, more static SRAM, no better confidence). **small2 does not Pareto-dominate current once accuracy is reported honestly as a five-seed statistic.** On mean accuracy, current (92.46%, 0/5 collapsed) exceeds small2 (78.83%, 1/5 collapsed) by a wide margin, because small2's headline mean absorbs a 20% collapse rate that current does not exhibit; small2 retains a real advantage in latency (7.67 ms vs. 10.66 ms) and model flash (3,040 B vs. 4,444 B), and its *converged-only* accuracy (93.95%) is still competitive with current's mean, but the binomial sampling uncertainty on a 1,507-sample test set alone is on the order of ±0.6–1.2 percentage points at these accuracy levels, so a ∼1.5-point converged-only gap on top of a demonstrated 20% risk of outright training failure does not support a dominance claim for small2 it supports treating small2 as a higher-variance, higher-risk alternative rather than a strictly better one. **large1 and large2 are the more defensible improvements over current** both post higher mean accuracy (93.71% and 94.08% vs. 92.46%) with visibly tighter standard deviations (0.61 and 0.38 points vs. 1.85) and zero collapses across their five seeds each, at the cost of higher latency and flash; this pattern simultaneously higher mean and lower variance is the kind of evidence a practitioner should weigh more heavily than small2's collapse-prone, high-ceiling profile, though $n = 5$ seeds per topology remains too small to claim formal statistical significance. **At the primary topology, int8's clearest advantage is flash** a 47% model-flash reduction and 8% less firmware flash than int16. It also shows a modest 416 μs reduction in the softmax-excluded `infer_us` timer (Section 7.3), but this advantage reverses once softmax is included in the latency budget, and it trails int16 on accuracy and confidence throughout making int8 Pareto-preferable specifically when flash headroom is the binding constraint, and Pareto-dominated by int16 on every other axis.

Fig. 10 draws the same seven configurations against the device's two hard resource ceilings directly the 2,048 B SRAM limit and the 30,720 B usable-flash limit with marker size scaled to accuracy (five-seed mean for the int16 topology rows, deployed-checkpoint accuracy for int8/float32, consistent with Table 17), unifying the streaming-feasibility result of Section 5 with the topology and precision trade-offs of Sections 7.2 and 7.7 in one view. The

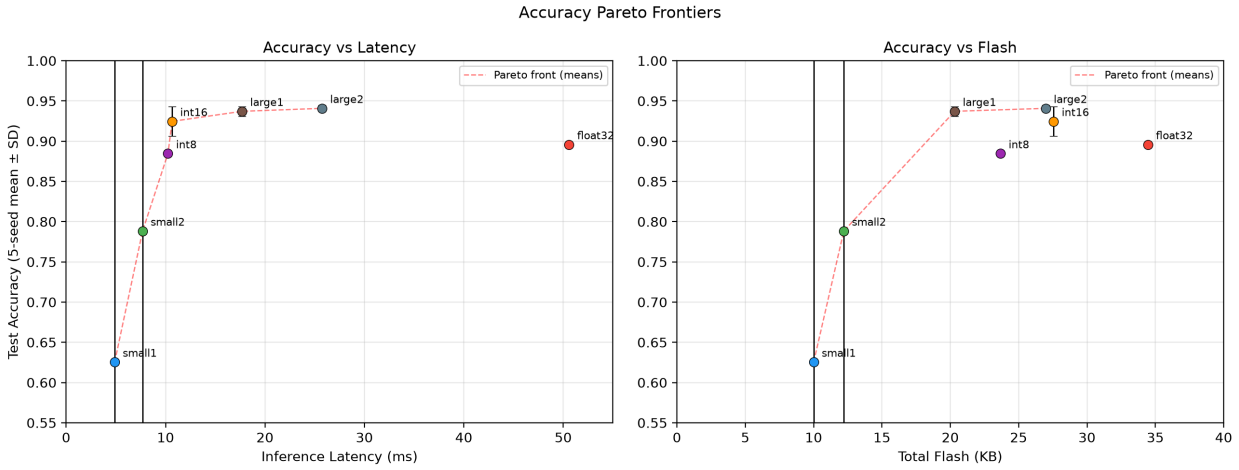

Figure 9: Pareto frontiers using five-seed mean ± SD accuracy for the int16 topology points (small1, small2, large1, large2, and current-as-"int16") and single deployed-checkpoint accuracy for int8 and float32 (no error bars, since precision was not swept across seeds). Left: accuracy vs. inference latency; right: accuracy vs. total firmware flash. Once seed fragility is accounted for, small2 no longer dominates current: current's tight, collapse-free mean (92.5% ± 1.9) sits close to small2's wide, collapse-affected mean (78.8% ± 33.8), with the Pareto front instead running through large1 and large2, whose higher means and much tighter SDs make them the more credible improvements over current at higher latency and flash cost. float32 remains dominated outright by int16; int8 sacrifices accuracy for a flash reduction alone.

dashed flash line in the figure represents the 30,720 B usable-application-flash budget after the bootloader reservation (Table 2), not the chip's raw 32,768 B physical capacity. Every configuration plotted here uses layer-streamed execution and therefore sits well inside the SRAM ceiling (678–1,222 B of 2,048 B, 33–60%); none approaches the flash ceiling either, with float32 the closest at 27,326 B (26.7 KB, its firmware-flash total from Table 18) against that 30,720 B usable-flash line, leaving it the largest of the seven but still comfortably under budget. This is the direct visual counterpart to Table 6, every point shown here failed to compile or crashed under conventional SRAM-resident staging at its respective topology, and appears in this feasible region only because of the execution-strategy change documented in Section 5, not because any of these models is inherently small enough to be a foregone conclusion on this device. Note that small1 and small2's comparatively small marker sizes in this figure reflect their lower collapse-inclusive mean accuracy, not a memory-footprint effect both remain the smallest-flash, smallest-SRAM configurations tested despite their diminished accuracy marker size.

8. Firmware Resource Usage

Table 18 lists whole-firmware resource consumption for the primary topology at all three precisions, against the 30,720 B usable-flash budget defined in Section 4.

Table 18: Firmware resource usage (percentages relative to 30,720 B usable flash and 2,048 B SRAM)

Resource	Float32	int16	int8
Flash	27,326 B (89%)	23,778 B (77%)	21,884 B (71%)
SRAM (global)	1,222 B (60%)	1,098 B (54%)	1,118 B (55%)
SRAM (free)	826 B (40%)	950 B (46%)	930 B (45%)

Table 18 isolates the primary topology; Fig. 11 extends the same SRAM and flash accounting across all seven configurations evaluated in this paper. The SRAM panel shows every configuration retains substantial free headroom below the 2,048 B ceiling, with the smallest margin (float32, 826 B free) still comfortably positive. The flash panel confirms the ordering already established in Table 17: int16 (23.2 KB) and float32 (26.7 KB) are the two largest firmware-flash consumers among the seven, both driven by the primary topology's larger weight matrices relative to small1/small2, while every configuration remains under the 30,720 B usable-flash ceiling with room to spare.

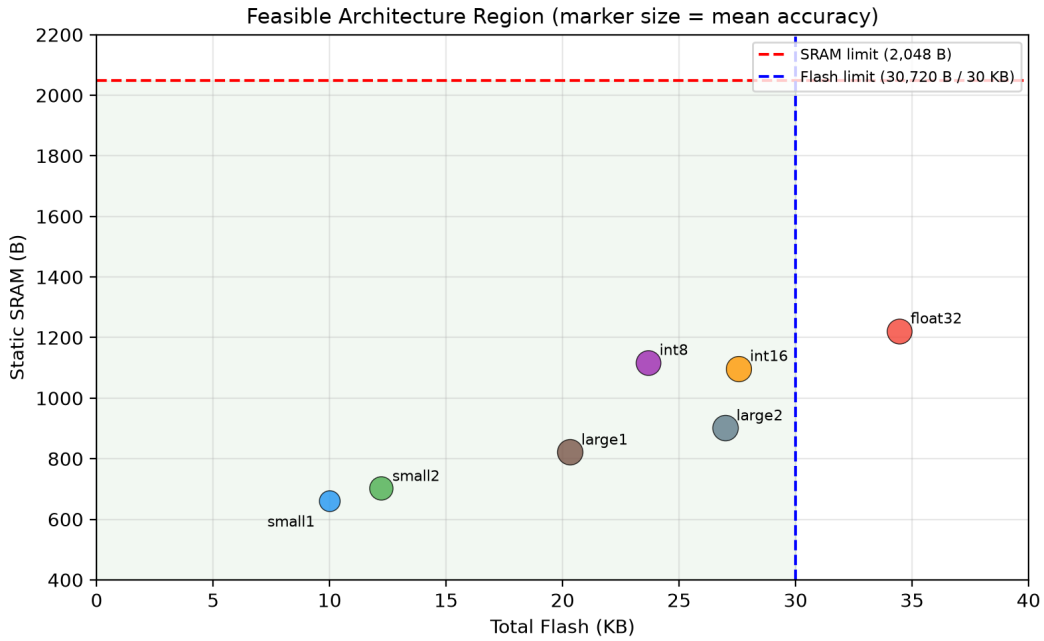

Figure 10: Feasible design space under layer-streamed execution: static SRAM vs. total flash for all seven configurations, marker size scaled to accuracy (five-seed mean for int16 topology points, deployed-checkpoint accuracy for int8/float32), against the device's 2,048 B SRAM limit and 30,720 B usable-flash limit (the flash limit reflects the post-bootloader application-flash budget from Table 2, not the chip's raw 32,768 B physical capacity). All seven configurations sit comfortably inside both limits under layer-streaming; Table 6 shows none of the corresponding topologies would appear in this region at all under conventional SRAM-resident weight staging. small1 and small2's smaller markers reflect their collapse-inclusive mean accuracy (Table 11), not a memory penalty.

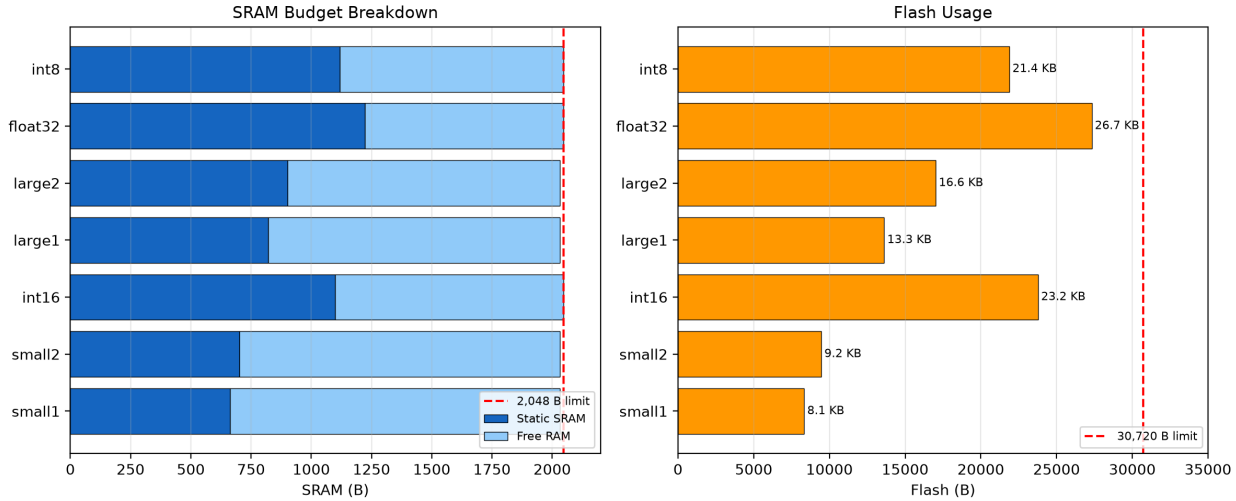

Figure 11: SRAM and flash accounting across all seven configurations. Left: static SRAM (dark) and free SRAM (light) against the 2,048 B ceiling; every configuration retains positive headroom. Right: total flash usage against the 30,720 B usable-flash ceiling, labeled in KB; int16 and float32 at the primary topology are the two largest consumers, and all seven configurations remain comfortably under budget.

9. Discussion

The results in Section 7 answer the primary research question posed in Section 1 more precisely than a single accuracy-versus-precision comparison could. Three findings, taken together, form the paper's design rule.

Execution strategy is a binary feasibility gate, not a tunable trade-off. Section 5 showed that conventional SRAM-resident weight staging is infeasible at every topology tested, including topologies far smaller than the primary model. This means the choice between conventional and streamed execution is not a point on a Pareto frontier alongside precision and topology; it is a precondition that must be satisfied before precision and topology choices are even meaningful on this device.

Once streaming is fixed, precision and topology form a genuine joint design space, and the five-seed replication (Section 6.4) revises which point in it is actually preferable to the primary model examined in depth here. A single-seed sweep in the original submission suggested that a smaller topology (small2) reached higher accuracy at lower latency and lower flash than the current model. Retraining every topology from five independent seeds does not confirm this, small2's collapse-inclusive mean accuracy (78.83%) is well below current's (92.46%), because small2 collapses to near-chance accuracy in 1 of 5 seeds a training-reliability risk current does not share (0/5 collapses). Small2's accuracy *when it does converge* (93.95%) remains competitive with current's mean, so small2 is better characterized as a higher-ceiling, higher-risk alternative than as a categorically superior one. The wider topologies, large1 and large2, present the more defensible case for outperforming current; both post higher mean accuracy with visibly tighter standard deviations and zero observed collapses, at the cost of additional latency and flash. Int8, independently of the topology question, buys flash headroom at a cost to accuracy, latency stability, and confidence that int16 does not pay. The current topology's central role in this paper's per-class, confidence, and confusion analysis (Sections 7.1–7.5) reflects that it was the originally deployed configuration used to develop the deeper analysis pipeline; the five-seed replication additionally clarifies that this specific deployed checkpoint (89.58%) underperforms current's own fresh-seed mean (92.46%), so it should be read as the fixed subject of this paper's detailed instrumentation rather than as the best accuracy current can achieve. Table 17 and Table 11 together are offered as the actionable design reference, a designer prioritizing minimum flash and latency with acceptable risk tolerance may reasonably choose small2 with awareness of its collapse rate and a plan to validate convergence before deployment; a designer prioritizing training reliability at a modest resource premium should prefer large1 or large2 over the primary current topology.

Confidence and accuracy are separate axes that can move independently, and both should inform a precision choice. Int16's live confidence distribution closely matches float32's; int8's live confidence distribution diverges more from int16's than its accuracy loss alone would suggest (Section 7.4). A firmware design that gates downstream actions on a confidence threshold could experience a larger reduction in its effective usable prediction rate under int8 than raw accuracy numbers alone would suggest, since Section 7.4's bucket-level reliability analysis also shows the model is systematically overconfident in its 40–80% confidence range under both precision groupings tested (float32/int16 and int8).

On-device inference optimization has a ceiling on total-system impact. Section 7.6 showed that host-side perception dominates end-to-end latency by more than 5× over on-device classification. This does not diminish the value of the precision and topology characterization in Sections 5–7.7, but it does mean that claims about the system's overall real-time responsiveness should be scoped to the classification stage specifically, as done throughout this paper.

Stated as a practical design rule for MLP inference on comparably constrained 8-bit MCUs, *verify execution-strategy feasibility first, independently of any particular model; treat precision and topology jointly rather than fixing one and sweeping the other; and evaluate confidence distribution alongside accuracy whenever quantization decisions feed a threshold-gated downstream action.*

10. Limitations

10.1. Subject evaluation with a pooled random split

All training and test data were collected from a single subject across multiple recording sessions with varied backgrounds, lighting, and hand orientations, then combined into one pool before a random 70/15/15 split (Section 6.2). This measures within-subject generalization under varied imaging conditions; it does not measure cross-subject generalization, since no session-independent or subject-independent holdout was constructed for this study. Performance on a different signer's hand shape, skin tone, or signing style is therefore untested and may differ substantially. This

remains an unresolved limitation of the present study, and no claim in this paper should be read as extending beyond the single-subject scope stated here.

10.2. Seed count and unresolved collapse mechanism

The original submission trained each of the five topologies in Section 7.2 once from a single random initialization and flagged this as an open question about whether the resulting ranking was a genuine architectural effect. Section 6.4 resolves this for the specific question originally posed retraining from five independent seeds shows that small2's apparent single-run advantage over current was not a stable architectural effect but a combination of a favorable seed and a demonstrated 20% collapse rate that current does not exhibit, while large1 and large2 show a more credible, tighter-variance advantage over current. A residual limitation remains five seeds per topology is enough to detect the collapse failure mode and produce a usable mean and standard deviation, but is too small a sample to place a reliable confidence interval on the collapse *rate* itself (e.g., distinguishing a true 20% collapse probability from a true 10% or 30% probability) or to support formal significance testing between topologies' converged accuracy; a larger seed count (e.g., $n = 20$–30) would be needed to characterize the collapse rate precisely and is left to future work. The mechanism behind the collapse itself plausibly a dead-ReLU or poor-initialization effect specific to narrower first-hidden-layer widths was not directly diagnosed (e.g., by inspecting activation statistics of collapsed runs) and is reported as an observed failure mode rather than an explained one.

10.3. Letter scope

The evaluation covers five letters (A–E) rather than the full 26-letter alphabet. Extending coverage would require additional output neurons and, plausibly, a wider hidden layer, increasing flash, SRAM, and computation requirements; the resulting effect on the int8-versus-int16 trade-off remains to be evaluated.

10.4. Dependency

The system depends on a host PC to run MediaPipe hand-landmark extraction; it is not a standalone device. Section 7.6 quantifies this dependency's latency cost directly.

10.5. Energy and power consumption

Instantaneous power and energy per inference were not measured. The present characterization is therefore limited to accuracy, confidence, latency, flash, and SRAM, and does not establish an energy-efficiency advantage between precision or execution strategies.

10.6. Softmax timing mechanism not fully isolated

Section 7.3's explanation for int8's anomalous softmax timing (data-dependent AVR `exp()` cost) is plausible and consistent with the measured pattern, but was not confirmed by directly logging output-score magnitudes alongside timing; this remains a plausible mechanism rather than an isolated, proven cause.

10.7. A→C confusion not resolved

Section 7.5 localizes the dominant error mode to overlapping hand-pose representations in activation space, but does not test an intervention that would confirm this diagnosis by resolving the confusion; it remains a characterization of the error mode rather than a fix.

11. Conclusion

This paper characterized the feasible design space of layer-streamed neural network inference on an ATmega328P, using a five-letter ASL hand-landmark classification task as a representative validation workload. Rather than presenting a single model at a single precision, it treated execution strategy, weight precision, and network topology as three jointly-evaluated design variables under a fixed 2,048 B SRAM and 30,720 B usable-flash budget. The contribution is a controlled characterization of how these deployment variables interact under an extreme SRAM constraint, rather than a new mechanism for reading weights from flash.

A streaming-ablation experiment established that conventional, SRAM-resident weight staging is infeasible compiler-rejected or hardware-crashed at every one of five tested topologies (869–4,997 parameters), making layer-streamed execution the only one of the two evaluated weight-staging strategies that executed successfully across the tested topology range. Within that constraint, int16 quantization matched float32 accuracy exactly (89.58%, 100% argmax agreement) while delivering a 4.7× inference speedup (10.66 ms vs. 50.59 ms), and produced a live confidence distribution closely matching float32's as well. Int8 traded 1.13 accuracy points, a lower mean confidence, and a wider confidence distribution for a further 47% reduction in model flash. Int8 reduced the softmax-excluded dense inference time (`infer_us`) by 416 μs relative to int16 (10.24 ms vs. 10.66 ms), but its substantially longer softmax stage (2.69 ms vs. int16's 0.98 ms) reversed this advantage when softmax was included in the latency budget, making int8 roughly 1.3 ms slower than int16 for softmax-inclusive on-device classification (12.93 ms vs. 11.64 ms). The five-seed topology replication further showed a seed-fragility failure mode in the two narrower topologies, with near-chance collapse in 2/5 and 1/5 seeds for small1 and small2, respectively, while current, large1, and large2 showed no collapse in five seeds each. The wider topologies therefore provide the more credible lower-variance accuracy improvements over current, although the small seed count does not support formal significance testing. End-to-end latency decomposition showed that host-side perception, not on-device classification, dominates total system latency by more than 5×, so the real-time contribution is specifically the classification stage.

Taken together, these results suggest a practical design rule for MLP inference on comparably constrained 8-bit microcontrollers; verify execution-strategy feasibility independently of any particular model before optimizing precision or topology, treat precision and topology as a joint rather than sequential design decision, and evaluate prediction-confidence distribution alongside accuracy whenever a quantization choice feeds a downstream threshold-gated action. Future work includes a larger-seed-count study (e.g., $n = 20$–30 per topology) to place a reliable confidence interval on the observed collapse rates and diagnose their mechanism, session and subject-independent evaluation to establish cross-subject generalization, extension to the full 26-letter alphabet, and a fully standalone variant replacing host-side MediaPipe with an on-board camera module.

CRediT authorship contribution statement

Al Mahmud Samiul: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code, including data collection, feature extraction, model training, quantized model export, streaming-ablation firmware, topology-scaling firmware, and Arduino firmware for float32, int16, and int8 inference, together with the trained model checkpoints for the five evaluated topologies and the dataset splits used in this study, are publicly available at `https://github.com/samiul000/asl-nano-mlp`.

Acknowledgements

The author received no specific funding for this work.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used Anthropic's Claude to assist with language refinement, proofreading for grammar and typography, manuscript consistency checks. After using this tool, the author thoroughly reviewed and edited the content as needed and takes full responsibility for the final content of the publication.

References

[1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device Real-time Hand Tracking," *arXiv preprint arXiv:2006.10214*, 2020.

[2] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[3] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[4] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference," *arXiv preprint arXiv:2103.13630*, 2021.

[5] R. David *et al.*, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," *arXiv preprint arXiv:2010.08678*, 2021. Also in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, pp. 800–811, 2021.

[6] C. Banbury *et al.*, "MLPerf Tiny Benchmark," *arXiv preprint arXiv:2106.07597*, 2021.

[7] H. A. Abushahla, D. Varam, A. J. N. Panopio, and M. I. AlHajri, "Neural Network Quantization for Microcontrollers: A Comprehensive Survey of Methods, Platforms, and Applications," *arXiv preprint arXiv:2508.15008*, 2025.

[8] K. Xia, W. Lu, H. Fan, and Q. Zhao, "A Sign Language Recognition System Applied to Deaf-Mute Medical Consultation," *Sensors*, vol. 22, no. 23, p. 9107, 2022, doi: 10.3390/s22239107.

[9] J. Y. Lim *et al.*, "A Sign Language Recognition System with Pepper, Lightweight-Transformer, and LLM," *arXiv preprint arXiv:2309.16898*, 2023.

[10] S. L. Kimpinde and P. O. Olukanmi, "Efficient Word-Level Sign Language Recognition Using Quantized Spatiotemporal Deep Learning for Low-Power Microcontrollers," *Algorithms*, vol. 19, no. 4, p. 248, 2026, doi: 10.3390/a19040248.

[11] A. T. Nguyen Ngoc, T. P. Truong, T. A. Nguyen Duong, V. L. Nguyen, and M. D. Phung, "Multimodal Smart Glove for Sign Language Recognition Using Deep Learning," *arXiv preprint arXiv:2607.06996*, 2026.

[12] S. Pangerd, S. Kommalest, S. Khiaophrom, and V. Kunbunjung, "IoT-Based Sign Language Recognition System," *Engineering Transactions: A Research Publication of Mahanakorn University of Technology*, vol. 29, no. 1, pp. 9–17, 2026.

[13] Arduino, "Arduino Nano Technical Reference," [Online]. Available: https://docs.arduino.cc/hardware/nano/, accessed 2026.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proc. 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

[15] V. K. Singh, "Memory-Efficient Neural Network Deployment Methodology for Low-RAM Microcontrollers Using Quantization and Layer-Wise Model Partitioning," in *2024 IEEE Pune Section International Conference (PuneCon)*, 2024, doi: 10.1109/PuneCon63413.2024.10895526.

[16] Y. A. Izotov, A. Velichko, and P. Boriskov, "Method for Fast Classification of MNIST Digits on Arduino UNO Board Using LogNNet and Linear Congruential Generator," *J. Phys.: Conf. Ser.*, 2021.